

Lab 5: A Kubernetes Deployment That Survives Failure

Declare it, expose it, break it deliberately, and watch the controller put it back.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

Two 2-hour blocks

SUBMIT

icclab5-<ROLLNO>.pdf

MARKS

15

OFFERING

Odd Semester 2026-27

Setup

```
kind create cluster --name icc          # or: minikube start
kubectl cluster-info
kubectl get nodes
```

A local cluster costs nothing and behaves like the real thing for everything in this lab. Use a managed cluster only if you have credit to spare.

Part 1: Declare it (4 marks)

1.1 Write a Deployment for the image you built in Lab 3, with 3 replicas, correct labels and selector, and `containerPort` declared. Load your image into the cluster (`kind load docker-image myapp:1.0`).

1.2 Apply it. Show `kubectl get pods -o wide` : three pods, on which nodes?

1.3 Delete one pod by name. Run `get pods` again immediately and after ten seconds. Explain what happened and which component did it.

1.4 Set `replicas: 5`, apply, and observe. Then set it back to 3. Explain in one sentence why this is *declarative* rather than *imperative*.

Part 2: Expose it (3 marks)

2.1 Add a Service selecting your pods, port 80 to targetPort 8080. Apply.

2.2 Show `kubect1 get endpoints`. How many endpoints, and why that number?

2.3 Change the Service selector to a label nothing carries. Show the endpoints list is now empty and the service is unreachable. Restore it. Explain what this tells you about how Services find pods.

2.4 Port-forward and confirm you can reach the application.

Part 3: Health (4 marks)

3.1 Add a `readinessProbe` and a `livenessProbe` on **different** paths.

3.2 Make the readiness endpoint start returning 503. Observe: does the pod restart? Does it still receive traffic? Show `kubectl get endpoints` before and after.

3.3 Now make the **liveness** endpoint fail. Observe the restart count with `kubectl get pods`. Show it climbing.

3.4 Explain, in three sentences, why making a liveness probe depend on the database is a mistake that turns a brief database problem into a total outage.

Part 4: Resources and scheduling (2 marks)

4.1 Deploy **without** resource requests. Then add requests and limits and redeploy. Use `kubectl describe node` to show how the scheduler's view changed.

4.2 Set a memory limit lower than the application needs. Show the pod being `OOMKilled` in `kubectl describe pod`. Explain the difference between exceeding a CPU limit and exceeding a memory limit.

Part 5: Rollout and rollback (2 marks)

- 5.1 Build `myapp:1.1`, load it, and update the Deployment's image. Watch `kubectl rollout status`.
- 5.2 Deploy a **broken** `myapp:1.2` whose readiness probe never passes. Show that the rollout stalls and that the old pods keep serving. Explain why this is the behaviour you want.
- 5.3 `kubectl rollout undo`. Confirm recovery.

Teardown

```
kubectl delete -f .  
kind delete cluster --name icc
```

What to submit

`icclab5-<ROLLN0>.pdf` : your manifests as text, the command output for each observation, the endpoints before and after in 3.2, the restart count in 3.3, the OOMKilled description, and the stalled rollout in 5.2.

MARKS	FOR
4	Part 1, especially the explanation of self-healing in 1.3
3	Part 2, especially the empty-endpoints demonstration
4	Part 3: both probe failures observed and correctly distinguished
2	Part 4: OOMKilled reproduced and explained
2	Part 5: the stalled rollout and rollback